

操作系统原理第五章作业

姓名：林宏宇 学号：23320093 截止日期：2025 年 04 月 15 日

完成日期：2025 年 04 月 04 日

Question 1: 为什么对调度程序而言, 区分 CPU 约束程序和 I/O 约束程序很重要?

Answer 1:

1.因为它们对系统资源的需求和模式不同, 合理的调度策略可以提高系统的整体性能和响应能力。

CPU 约束程序主要进行计算密集型任务, 用于大批量的计算, 它们的通常会长时间占用 CPU。调度器需要确保程序得到足够的 CPU 时间来运行, 并且减少等待时间, 避免过多的上下文切换。

I/O 约束程序主要进行 I/O 密集型操作, 长时间等待 I/O 设备进行数据传输。等待 I/O 时并不需要占用 CPU, 因此调度器可以将 CPU 资源分配给其他程序, 但需要进行合理的上下文切换响应 I/O 操作。

2.通过区分这两类程序, 调度器可以优化资源的使用, 同时给每个程序分配足够资源。

当 CPU 约束程序需要资源时, 而 I/O 约束程序可以让出 CPU 资源, 在等待 I/O 时切换到其他任务。

当系统中存在大量 I/O 约束程序时, 在等待 I/O 时会有空闲时间, 调度器可以利用空闲时间来调度其他进行避免 CPU 资源浪费。

CPU 约束程序占用过多时间会导致 I/O 约束程序长期无法获得 CPU, 即产生饥饿现象, 应该合理调度 CPU 约束程序, 确保所有程序都能分配到资源。

Question 2: What are some advantages of peer-to-peer systems over client-server systems?

考虑下面一组进程,进程占用的 CPU 区间长度以毫秒来计算:

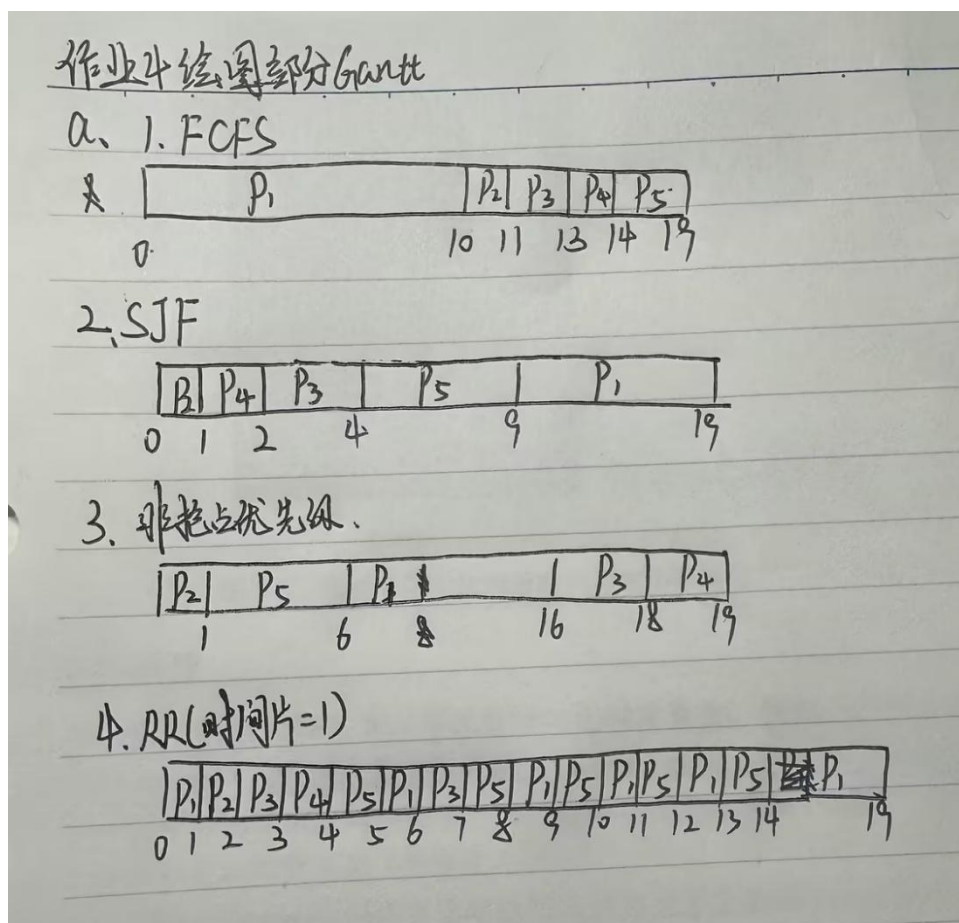
进程	CPU 区间时间 (毫秒)	优先级
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

假设在 0 时刻进程以 P1、P2、P3、P4、P5 的顺序到达。

- 画出 4 个 Gantt 图分别演示使用 FCFS、SJF、非抢占优先级(数字越小代表优先级越高)和 RR(时间片=1)算法调度时进程的执行过程。
- 每个进程在每种调度算法下的周转时间是多少?
- 每个进程在每种调度算法下的等待时间是多少?
- 哪一种调度算法的平均等待时间最小(对所有的进程)?

Answer 2:

A. 画出 4 个 Gant 图



B. 每个进程在每种调度算法下的周转时间

周转时间是指一个进程从进入系统到执行完成并退出系统的总时间

FCFS:

P1: 完成时间=10, 周转时间=10

P2: 完成时间=11, 周转时间=11

P3: 完成时间=13, 周转时间=13

P4: 完成时间=14, 周转时间=14

P5: 完成时间=19, 周转时间=19

SJF:

P2: 完成时间=1, 周转时间=1

P4: 完成时间=2, 周转时间=2

P3: 完成时间=4, 周转时间=4

P5: 完成时间=9, 周转时间=9

P1: 完成时间=19, 周转时间=19

非抢占优先级调度:

P2: 完成时间=1, 周转时间=1

P5: 完成时间=6, 周转时间=6

P1: 完成时间=16, 周转时间=16

P3: 完成时间=18, 周转时间=18

P4: 完成时间=19, 周转时间=19

RR (时间片=1):

P1: 完成时间=19, 周转时间=19

P2: 完成时间=2, 周转时间=2

P3: 完成时间=7, 周转时间=7

P4: 完成时间=4, 周转时间=4

P5: 完成时间=14, 周转时间=14

C. 每个进程在每种调度算法下的等待时间

等待时间 = 周转时间 - 执行时间

FCFS:

P1: 等待时间=10-10=0

P2: 等待时间=11-1=10

P3: 等待时间=13-2=11

P4: 等待时间=14-1=13

P5: 等待时间=19-5=14

SJF:

P2: 等待时间=1-1=0

P4: 等待时间=2-1=1

P3: 等待时间=4-2=2

P5: 等待时间=9-5=4

P1: 等待时间=19-10=9

非抢占优先级调度:

P2: 等待时间=1-1=0

P5: 等待时间=6-5=1
P1: 等待时间=16-10=6
P3: 等待时间=18-2=16
P4: 等待时间=19-1=18
RR (时间片=1):
P1: 等待时间=19-10=9
P2: 等待时间=2-1=1
P3: 等待时间=7-2=5
P4: 等待时间=4-1=3
P5: 等待时间=14-5=9

D. 哪一种调度算法的平均等待时间最小

FCFS: 平均等待时间 = $(0+10+11+13+14) / 5 = 9.6$

SJF: 平均等待时间 = $(0+1+2+4+9) / 5 = 3.2$

非抢占优先级调度: 平均等待时间 = $(0+1+6+16+18) / 5 = 8.2$

RR (时间片=1): 平均等待时间 = $(9+1+5+3+9) / 5 = 5.4$

平均等待时间最小的调度算法是 SJF, 平均等待时间为 3.2 毫秒。

Question 3:

下面哪种调度算法能导致饥饿?

a. 先到先服务 b. 最短作业优先 c. 轮转法 d. 优先级

Answer 3:

答案是: b. 最短作业优先 和 d. 优先级。

a. 先到先服务

按请求到达的顺序调执行任务, 不会导致某些任务永远无法执行, 则不会造成饥饿现象。

b. 最短作业优先

优先调度执行时间最短的作业。对于执行时间非常长的作业, 如果不断产生最短作业的话, 执行时间很长的作业可能会被持续推迟, 得不到调度, 形成饥饿现象。

c. 轮转法

通过固定时间片轮流调度所有任务, 因为每个进程都会被定期分配到 CPU, 不会导致饥饿。

d. 优先级

优先调度优先级高的任务。对于优先级很低的任务, 可能会持续的因为高优先级而被一直推迟, 一直无法执行, 形成饥饿现象。